
Analysis of Energy Consumption on Small RAG LLM Models

Christopher Barbosa ^{* 1} Sean Jenkins ^{* 1}

Abstract

The field of large language models (LLMs) is fast-growing, as the size of models increases, so do the energy requirements for training and running them. Today, LLM systems geared towards inference and QnA generative interactions can be most prominently seen in models like GPT, Grok, etc. Retrieval augmented generation (RAG) is a tool that supplies the LLM with the ability to retrieve external information, aggregate it with its training solution, and return a stronger, more accurate response. However, this comes at a cost, as models that utilize RAG to any extensive degree can perform very slowly and require a lot of energy to compute. In this paper, we use small models like GPT-2 and Gemma3-4b, measuring the energy consumption, solution accuracy, and latency. We found that although the Gemma-3-4b model was the most accurate, it was also the most costly on energy. In the end, we speculate towards the use of lightweight RAG-based distributed VLM systems, which have the capacity to provide both high accuracy and low energy consumption. Code for the project is hosted here: [\[GITHUB\]](#)

1. Introduction

Over the past several decades, we have seen an astronomical increase in innovation in the tech sphere, largely due to the advancements in the field of artificial intelligence (AI). The ability for AI systems to process, synthesize, and draw conclusions on large amounts of data allows humans to delegate tasks and defer to AI for information gathering. The magic of AI lies in its statistical learning and inference capabilities, which allow it to synthesize data and make decisions with the highest probability of success based on its training set. Consequently, the current models of AI are developed, trained, and finetuned on gigantic datasets that take up a lot of space and time to learn from [\[\(Feigenoff, 2025\)\]](#). This upwards trend extends just past the memory consumption of the models and applies to energy as well. In areas where inference and problem solving are the focus for the AI system, in order to fully process and generate an accurate response, the energy requirement for the system is

very high [\[\(Feigenoff, 2025\)\]](#).

A direct and most pressing example of this is the area of large language models (LLMs). The focus of LLMs lies in natural language processing (NLP), inference, and chatbot capabilities. As the name suggests, LLMs utilize access to and are trained on large datasets to return as accurate information as possible; as with examples like ChatGPT, GEMINI, etc., the versatility and effectiveness of such models are vast. The first LLM models were developed through the ideas on artificial neural networks and were used in more classification and task problems. Later on, we saw developments towards statistics-based models which utilize practices in machine learning to increase the accuracy and problem-solving capabilities [\[\(RAIAAN et al., 2024\)\]](#). However, with each new model or advancement seen, we see an increase in memory and energy consumption as the size of data and number of parameters increase. This leads us into a more recent development in LLMs, retrieval augmented generation (RAG) models.

The main difference between traditional LLM models and those utilizing RAG is an aspect of external retrieval of information. In training an LLM model, the data that they are provided is all they see or pull from, and so the quality and/or size of the training dataset can, in turn, tell you the quality and accuracy of the LLM model. Consequently, in times of poor training or insufficient datasets, a common practice performed is called finetuning, where the model is retrained or given larger datasets, and/or more parameters are added to its inference function. The critical point of RAG is that the LLM has access to an external well of information that, during the inference phase, it can pull from in order to enhance its response. However, RAG systems are notoriously slow and require a lot of computing power (which can be improved with the use of VLM or distributed paradigms). This paper serves to look into the utilization of RAG LLM models in more domain-specific circumstances and analyze the energy consumption. We go on throughout the remainder of this paper to discuss the methodology and analysis metrics we employed, our experimental results and findings, as well as the conclusions and directions for further study based on our results.

1.1. Goal of this Paper

Through this paper and the research, we especially wanted to deepen our knowledge of AI systems and develop our research capabilities. We found that the area of energy analysis for LLMs is a topic that is greatly important. There is a clear trade-off in the space that, for optimized accuracy, you need to sacrifice computational power and energy consumption. However, as the trend turns towards models with hundreds of billions of parameters and lengthy training and fine-tuning periods, the energy required is becoming unreasonable. The aim is to allow for models to be light-weight and energy efficient while still maintaining a certain threshold of accuracy. The goal of this research was to look into the effectiveness of small models when aided with retrieval-augmented generation capabilities, as well as provide direction in the field towards the utilization of distributed architectures for such models.

2. Method and Metrics

To conduct our experimentation, we utilized three lightweight RAG models, namely: HF-RAG(GPT-2 with custom retrieval), Facebook-RAG (rag-token-base with DPR encoder), and lastly LangChain-RAG(gemma3-4b base model with FIASS retrieval).

2.1. Models and Technology

To be more specific, the custom retrieval feature for the GPT-2 model enhances its ability to generate accurate and relevant responses. Moreover, it allows it to retrieve information from external sources or pull from a domain-specific well of knowledge that it may not have trained on. This is perfect for our purposes as it enhances its ability to interact with the RAG architecture. It should also be noted that standard GPT-2 is not known to be good at inference or data retrieval, so these augmentations are necessary for the accuracy of our experiment.

A RAG-token-base is a non-finetuned implementation of the retrieval-augmented generation(RAG) architecture, geared towards and designed primarily for natural-language processing(NLP) tasks. It houses three key components: a question encoder, a retriever, and a generator. The dense passage retrieval(DPR) encoder encompasses two major functions, query encoding and passage encoding. Utilizing neural network architecture, the encoder represents texts as dense vectors, allowing for semantic similarities between queries and solutions to be found much more efficiently and computationally light. We decided to utilize such a model because it can be seen that for light-weight RAGs, RAG-token-based systems can actually increase the accuracy of the LLM and subsequently reduce the risk of outdated knowledge or hallucinations[(Cuconasu et al., 2024)]

Lastly, Gemma-3-4b is a relatively small Gemma-3 model that has seen very good results in inference and retrieval tasks. The model makes use of multi-modal systems, which allow it to process many different types of data and provide sufficient accuracy and low energy tradeoffs. The FAISS method of retrieval utilizes vector databases, which allow for quick retrieval and processing capabilities. This model served as our type of baseline compared to the other models, as it was relatively bigger, and the literature tells us that it should perform the best in terms of accuracy.

We coded our models and experiment benchmarks in Python, utilizing libraries like PyTorch and Transformers, in Google Colab with a Tesla T4 GPU. We designed each query to retrieve the top 5 most relevant documents, aggregate them, and then return a response. We will next discuss the metrics used for our experimentation and what we predict the results will be.

2.2. Metrics and predictions

The three metrics that we decided to focus on were energy(Jules or watt-hour), latency(seconds, end-to-end), and accuracy for a given query. As you can see in the following code implementation:

```

1 def run(self, query, docs):
2     prompt = query + '\n' + '\n'.join(
3         docs[:3])
4     t0 = time.time()
5     energy = 0.0
6     if NVML_AVAILABLE:
7         h = pynvml.
8             nvmlDeviceGetHandleByIndex
9             (0)
10        p0 = pynvml.
11            nvmlDeviceGetPowerUsage(h)
12            / 1000.0
13        out = self.generator(prompt)
14        latency = time.time() - t0
15        if NVML_AVAILABLE:
16            p1 = pynvml.
17                nvmlDeviceGetPowerUsage(h)
18                / 1000.0
19            energy = (p0 + p1) / 2 *
20                latency
21        return out, latency, energy

```

Listing 1. Rag-Runner code

We calculated energy using the following formula:

$$E_{\text{query}} = \frac{(p_o + p_1)}{2 \cdot \text{latency}}$$

Where p_o is the initial power usage of the model before the query, which takes in the argument of the device index at time 0, and p_1 is the power usage after the query is done. We also see that latency is defined by the equation:

$$\text{latency} = t_1 - t_0$$

i.e, it is time after the query minus the time before(utilizing the standard 1-D Euclidean metric of distance). Standardizing these values for all three models allows us to draw concrete conclusions as to the energy usage and latency.

For accuracy, we used both exact match and F1-score for the models. In order to be as rigorous as possible, we needed to employ normalization techniques on our inputs. In general, when we want to test the accuracy of a model, we do so by comparing the response generated vs the known ground truth text; however, how is this comparison processed, and/or what are the semantics that dictate what is close to the ground truth or not? Normalization demystifies this process. What we did was that we processed all texts by converting everything to lower case, removing all punctuation and spaces (to look at the raw characters), and then did a comparison from there. Moreover, we can define this as a function $norm()$ which takes in text as input; In this way, we can test exact-match, denoted by the function $em(,)$ of two inputs, which works as follows, for inputs p and gt (for model prediction and ground truth, respectively):

$$em(norm(p), norm(gt)) = \begin{cases} 1 & \text{if inputs are exactly equal} \\ 0 & \text{otherwise} \end{cases}$$

This provides a metric for correctness, but doesn't tell us anything about whether the returned solution was very close to the answer but slightly off(semantically accurate but lexically off). This is why we use the F1 score as well, where the formula is defined as follows:

$$F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Precision measures how much of the model's response is correct, and recall measures how close the response is to the ground truth. These two metrics allow for a more holistic measure of accuracy, which is important for our experimentation. For reference, we provide the code below of the implementation described:

```

1 def exact_match(pred, truth):
2     return int(normalize_text(pred) ==
3               normalize_text(truth))
4
5 def f1_score(pred, truth):
6     p = normalize_text(pred).split()
7     t = normalize_text(truth).split()
8     if not p or not t: return 0.0
9     common = set(p) & set(t)
10    if not common: return 0.0
11    precision = len(common) / len(p)
12    recall = len(common) / len(t)
13    return 2 * precision * recall / (
14        precision + recall)
    
```

Listing 2. Acc/EM code

From the background research and knowledge of the field, we predict that the models could perform very well on energy consumption, but there could be trade-offs for accuracy, as we limited the retrieval scope to only return the top 5 most relevant pieces of data. In the following section, we use the above metrics to compare the models given a small dataset to pull from.

3. Findings

The dataset that we used for our experiment is a RAG-instruct from FreedomIntelligence; the given dataset is designed to enhance the RAG capabilities for LLM models and improve overall performance. The creators specifically wanted a general-purpose RAG-instruct dataset that pulls data based on a source corpus(primarily data from Wikipedia), and has seen great results in both training and effective experimentation.[(Liu et al., 2024)]. As the dataset as a whole is very large at about 40K instructions, for our purposes and our limited computational power, we used a subset of 500 QnA responses to query our models with. In the following section, we will display our experimentation and discuss the results we found.

3.1. Results

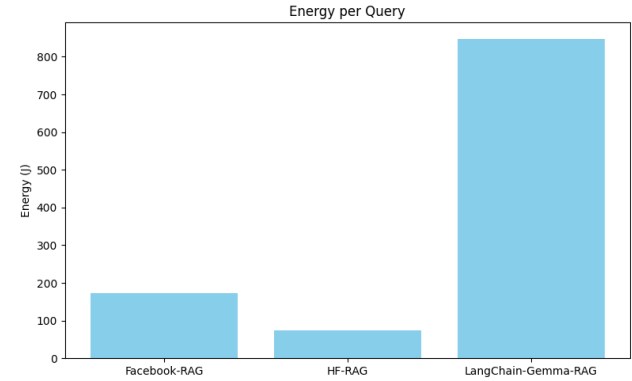


Figure 1. Energy Comparison Figure

Figure 1 presents the average energy consumption per query for the three evaluated RAG systems. HF-RAG demonstrates the lowest energy usage at approximately 73.8 joules per query, followed by Facebook-RAG at 172.0 joules, while LangChain-RAG consumes significantly more energy at 848.1 joules per query. This substantial difference can be attributed to architectural complexity and model size. As described previously, HF-RAG relies on a smaller GPT-2 backbone, resulting in minimal inference overhead. Facebook-RAG introduces additional energy costs due to dense passage retrieval(DPR) and token-level generation. LangChain-RAG incurs much higher energy consumption

due to its larger parameter count and more computationally intensive retrieval and generation pipeline. These results suggest that energy cost scales sharply with both model size and retrieval architecture.

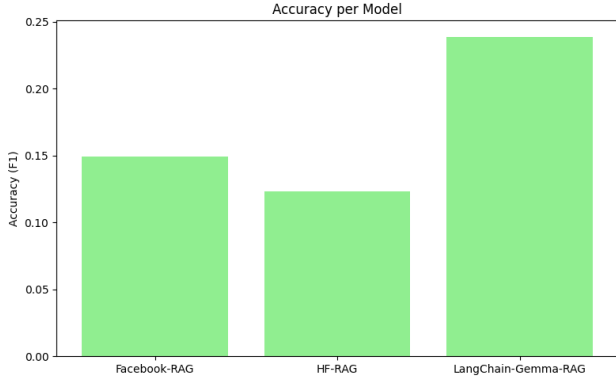


Figure 2. Accuracy Comparison Figure

Figure 2 displays the comparative results of accuracy (using Exact Match, EM) and F1-score for each model. All models achieved an EM score of 0.0, indicating that none consistently produced answers that exactly matched the normalized ground truth responses. This outcome, as theorized in the metrics section, highlights the limitations of EM for evaluating generative models. The experimentation suggests that the F1-score provides a more nuanced assessment of performance. LangChain-RAG achieved the highest F1-score at 0.239, followed by Facebook-RAG at 0.150 and HF-RAG at 0.123. The superior F1 performance of LangChain-RAG suggests that stronger retrieval mechanisms correlate with higher accuracy at capturing relevant semantic information; it is unknown whether the size of the model had any effect. However, this accuracy gain comes at the expense of significantly higher compute and energy costs.

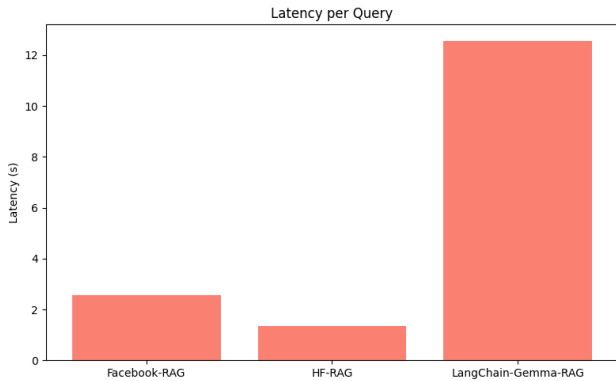


Figure 3. Latency Comparison Figure

Lastly, the third figure illustrates the average end-to-end latency per query for each model. HF-RAG exhibits the lowest latency at 1.34 seconds, making it the most responsive system tested. Facebook-RAG follows with a latency of 2.56 seconds, while LangChain-RAG shows substantially higher latency at 12.57 seconds. The increased latency of LangChain-RAG is consistent with its larger model size and increased complexity in retrieval mechanisms. These results demonstrate that retrieval-augmented generation (RAG) can introduce nontrivial latency overhead, particularly when combined with large backbone models and vector-based retrieval systems, which can in turn limit reliability for real-time or resource-constrained applications.

3.2. Discussion

model	latency_s	energy_j	em	f1
Facebook-RAG	2.556628	172.022590	0.0	0.149507
HF-RAG	1.340784	73.784435	0.0	0.123021
LangChain-RAG	12.574134	848.121443	0.0	0.238998

Table 1. Metrics Results Table

When analyzing the energy consumption, accuracy, and latency results, a clear trade-off emerges between efficiency and performance. HF-RAG is the most energy-efficient and fastest model; however, the trade-off is that it provides the lowest accuracy. LangChain-RAG achieves the highest F1-score, yet it does so at the cost of dramatically increased energy usage and latency. Facebook-RAG occupies a happy middle ground, displaying results of moderate energy consumption, latency, and accuracy. In the context of the broader field, these findings reinforce the growing concern that improvements in LLM accuracy (particularly through RAG) often come with unsustainable computational costs. For our experimentation, this highlights the potential use of lightweight RAG systems for efficiency-focused use cases, while utilizing larger RAG pipelines in scenarios where accuracy is the priority. This is a complex problem with no single optimal configuration; instead, RAG system design must be guided by application-specific constraints and circumstances.

4. Conclusion

This project investigated the trade-offs between energy consumption, latency, and accuracy in small retrieval-augmented generation (RAG) language models. By benchmarking HF-RAG, Facebook-RAG, and LangChain-RAG on a subset of the RAG-instruct dataset, we showed that improved semantic accuracy can be closely associated with increased inference latency and energy usage. LangChain-RAG achieved the highest F1-score but incurred substantial computational cost, while HF-RAG provided the lowest

latency and energy consumption at the expense of answer quality. These results emphasize that RAG system design involves inherent trade-offs between efficiency and performance.

Several limitations of this study should be acknowledged. First, as mentioned previously, the dataset used for evaluation was relatively small (consisting of only 500 query–answer pairs), which greatly limits the scope of data it could pull from. Second, retrieval depth was restricted to the top five documents, potentially constraining contextual coverage, particularly for smaller models. Third, prompt construction was intentionally simple, relying on direct concatenation of the query and retrieved documents; more structured or reasoning-oriented prompt templates could have shown great improvements in accuracy. Fourth, the dataset focused on general question-answering, which may not reflect performance in specialized or domain-specific tasks where RAG systems often excel. Finally, energy measurement was simplified to GPU power usage only, excluding CPU utilization, memory access, and retrieval overhead, which likely underestimates the true system-level energy consumption.

Despite these limitations, the findings provide meaningful insight into the energy–accuracy–latency trade-offs present in RAG-enabled LLMs and highlight the importance of incorporating energy efficiency as a core evaluation metric. A promising direction for addressing these challenges is the use of distributed models. Distributed RAG architectures decompose retrieval, reasoning, and generation across multiple nodes or cloud–edge systems, enabling parallel execution and improved resource utilization. By offloading lightweight retrieval or pre-processing tasks to edge devices and reserving energy-intensive generation for centralized servers, distributed models can reduce redundant computation and lower per-query energy costs while maintaining high accuracy [Li et al., 2025a].

In conclusion, while larger RAG systems currently offer superior semantic performance, their energy demands raise sustainability concerns. Distributed and lightweight RAG architectures present a compelling path forward for building scalable, energy-efficient language systems that better balance accuracy, latency, and environmental impact.

5. Impact Statement and Future Research Directions

Despite the small scale of this research, there can still be conclusions made about future experimentation. Our given study can serve as an example of how experimentation can be done and the metrics needed to draw similar conclusions. The impact of our research might be small when it comes to the broader field of machine learning and/or LLM

analysis; however, it highlights a pressing issue in energy consumption that, up till now, is an underdeveloped vein of experimentation. Due to limited computational power, we could not take full advantage of the RAG-instruct dataset; this could be a potential avenue of future research.

5.1. Distributed Models

As mentioned prior, the utilization of distributed VLM models and distributed techniques can be very useful in the field of retrieval-augmented generation LLMs. Future directions on RAG energy efficiency can be geared towards testing distributed models. Do they perform as well, or are there trade-offs? In most cases, distributed architectures can require more baseline energy. Does this contribute to the models’ consumption? Is there a way to eliminate this and still maintain accuracy? These questions and more provide for a rich direction of research that can potentially lead to better and more efficient RAG models.

5.2. Scalability and Efficiency

For our research, we focused on small models with a limited dataset. Can our results/line of experimentation be scaled up to larger models and still find the same results? Is there necessarily a correlation between energy and accuracy? Can we fit smaller models that perform the same as larger ones but consume much less energy? These types of questions are very relevant today as the demand for LLM models has never been higher, and the current trend is scaling the models upwards, with more parameters, more energy, and longer training and fine-tuning phases. This direction of research could uncover a different facet of RAG systems, and potentially allow companies to use smaller, more energy-efficient models and still see the same accuracy results.

References

- Cuconasu, F., Trappolini, G., Tonellotto, N., and Silvestri, F. A tale of trust and accuracy: Base vs. instruct llms in rag systems, 2024. URL <https://arxiv.org/abs/2406.14972>.
- Desislavov, R., Martínez-Plumed, F., and Hernández-Orallo, J. Trends in ai inference energy consumption: Beyond the performance-vs-parameter laws of deep learning. *Sustainable Computing: Informatics and Systems*, 38:100857, 2023. ISSN 2210-5379. doi: <https://doi.org/10.1016/j.suscom.2023.100857>. URL <https://www.sciencedirect.com/science/article/pii/S2210537923000124>.
- Feigenoff, C. An introduction to artificial intelligence. Technical report, Virginia Academy of Science, Engineering, and Medicine, Richmond, VA 23219, 2025.

- Li, J., Xu, C., Jia, L., Wang, F., Zhang, C., and Liu, J. Eacorage: Towards distributed tiered llm deployment using edge-assisted and collaborative rag with adaptive knowledge update, 2025a. URL <https://arxiv.org/abs/2410.20299>.
- Li, Y., Gumaste, D., and Mehmet Kerem Turkcan, e. a. Distributed vlms: Efficient vision-language processing through cloud-edge collaboration. *Columbia*, 1(1):1–7, 2025b.
- Liang Zhang, Z. c. Opportunities of applying large language models in building energy sector. *Renewable and Sustainable Energy Reviews*, 214(115558):N/A, 2025.
- Liu, W., Chen, J., Ji, K., Zhou, L., Chen, W., and Wang, B. Rag-instruct: Boosting llms with diverse retrieval-augmented instructions, 2024. URL <https://arxiv.org/abs/2501.00353>.
- Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., and Mian, A. A comprehensive overview of large language models, 2024. URL <https://arxiv.org/abs/2307.06435>.
- Oche, A. J., Folashade, A. G., Ghosal, T., and Biswas, A. A systematic review of key retrieval-augmented generation (rag) systems: Progress, gaps, and future directions, 2025. URL <https://arxiv.org/abs/2507.18910>.
- Ofosu-Ampong, K. Artificial intelligence research: A review on dominant themes, methods, frameworks and future research directions. *Telematics and Informatics Reports*, 14:100127, 2024. ISSN 2772-5030. doi: <https://doi.org/10.1016/j.teler.2024.100127>. URL <https://www.sciencedirect.com/science/article/pii/S2772503024000136>.
- RAIAAN, M. A. K., MUKTA, M., and KANIZ FATEMA, e. a. A review on large language models: Architectures, applications, taxonomies, open issues and challenges. *IEEE access*, 12(N/A):26839–26874, 2024.
- Wan, Y., Chen, Z., Liu, Y., Chen, C., and Packianather, M. Empowering llms by hybrid retrieval-augmented generation for domain-centric q&a in smart manufacturing. *Advanced Engineering Informatics*, 65:103212, 2025. ISSN 1474-0346. doi: <https://doi.org/10.1016/j.aei.2025.103212>. URL <https://www.sciencedirect.com/science/article/pii/S1474034625001053>.
- Wang, J., Fu, J., Wang, R., Song, L., and Bian, J. Pike-rag: specialized knowledge and rationale augmented generation, 2025. URL <https://arxiv.org/abs/2501.11551>.
- Wu, H., Wang, X., and Fan, Z. Addressing the sustainable ai trilemma: a case study on llm agents and rag, 2025. URL <https://arxiv.org/abs/2501.08262>.
- Yu, S., Tang, C., Xu, B., Cui, J., Ran, J., Yan, Y., Liu, Z., Wang, S., Han, X., Liu, Z., and Sun, M. Visrag: Vision-based retrieval-augmented generation on multi-modality documents, 2025. URL <https://arxiv.org/abs/2410.10594>.